**UNIVERSIDAD VERACRUZANA**
**FACULTAD DE ESTADÍSTICA E INFORMÁTICA**
**LICENCIATURA EN INGENIERÍA DE SOFTWARE**
Diseño de Software

# Documento de Arquitectura de Software (SAD)

Plantilla del proyecto final · versión ligera 5.0

| | |
|---|---|
| **Proyecto** | Bastión |
| **Equipo e integrantes** | Equipo 4 · Ángel Gabriel Aguilar Hernández · José Eduardo Prior Hernández |
| **Versión del documento** | v0.2 |
| **Entrega** | Primera entrega |
| **Fecha** | 12/10/2026 |

### Cómo usar esta plantilla

**1. Llenen lo que corresponde a la entrega en curso (primera entrega o entrega final). Lo demás puede quedar vacío por ahora.**

**2.** Documenten a fondo sólo lo que obliga a la arquitectura a tomar cierta forma. Si algo no aplica, díganlo en una línea y expliquen por qué.

**3.** Usen sólo siete identificadores: **QAS** (escenario de calidad), **ASR**, **ADR**, **C** (módulo o componente), **V** (vista), **EV** (evidencia) y **R** (riesgo o pendiente). Todo lo demás se nombra.

**4.** Cada decisión se escribe una sola vez, en su ficha de la sección 3. Las demás secciones la citan por su número de ADR.

**5.** Antes de empezar, lean el ejemplo resuelto del Sistema de Gestión Hotelera: muestra cada sección bien llenada.

**6.** Esta plantilla pide lo necesario para un trabajo logrado; lo que distingue un trabajo sobresaliente está en la rúbrica. Si algo aquí contradice el Manual o la Evaluación vigentes, mandan esos documentos. Si ya empezaron con otra versión de la plantilla, trasladen su contenido: el formato anterior no se recalifica.

**La cadena que el documento debe permitir seguir:** escenario, función o restricción → **ASR** → **ADR** → elemento **C** → vista **V** → evidencia **EV** → riesgo residual **R**

# 1. Control y glosario Primera entrega

*Mantengan esta sección breve y actualícenla en cada entrega.*

## 1.1 Control e historial

| | |
|---|---|
| **Sistema** | Bastión: juego de escritorio cliente-servidor en el que dos o cuatro jugadores mueven su peón y colocan muros por turnos para llegar primero a la orilla opuesta del tablero, con cuentas, salas privadas, amigos, chat moderado por reportes y ranking. |
| **Propósito del documento** | Explicar qué obliga a la arquitectura a tomar su forma, qué decisiones se tomaron y con qué alternativas se compararon, para que el equipo construya sin romperlas y el profesor pueda evaluarlas. |
| **Audiencia** | Los dos integrantes del equipo (para construir y mantener el sistema con un criterio común) y el profesor de la experiencia educativa (para evaluar y para confirmar las decisiones abiertas). |
| **Alcance** | Cubre el cliente de escritorio, el servidor de partidas, los contratos de comunicación y la base de datos, con las funciones de los 28 casos de uso del alcance (CU-01 a CU-28). Quedan fuera los 23 requisitos deseables (DES-01 a DES-23: entre ellos la IA, los espectadores, la tienda y los moderadores), el diseño gráfico y sonoro, el contenido de los catálogos (modos y palabras prohibidas) y la operación en producción: el sistema se despliega en las máquinas del equipo y en el aula. |

| Versión | Fecha | Cambios principales |
|---|---|---|
| v0.1 | 05/10/2026 | Primera entrega: control, glosario, drivers (contexto, interesados, funciones clave, restricciones, escenarios de calidad y ASR), decisiones ADR, módulos iniciales, vistas V-01 y V-02, y riesgos iniciales. |
| v0.2 | 12/10/2026 | Ajuste al recorte del alcance a 28 casos de uso (D-25 a D-29): referencias con la numeración nueva; sin IA, espectadores, tienda ni moderadores; Jugador e Invitado como únicos actores (V-01); se retira STK-12; QAS-07 pasa a la sanción automática por reportes y QAS-12 a la caída del servicio de correo. ADR-007 cifra el canal con TLS y sustituye a ADR-004; R-01 se reformula y se agrega R-09. |

## 1.2 Resumen de la arquitectura

*Máximo una página: problema, drivers principales, estructura general, decisiones esenciales y riesgos abiertos. Conviene escribirlo al final de cada entrega.*

**Problema.** Bastión es un juego de mesa por turnos inspirado en Quoridor, con partidas en línea de dos o cuatro jugadores, emparejamiento clasificatorio con elo, salas privadas con código, amigos, chat moderado por reportes y ranking (CU-01 a CU-28). La IA, los espectadores, la tienda y los moderadores quedan como requisitos deseables (D-25). Lo construye un equipo de dos personas, una en macOS arm64 y otra en Windows, con un stack propio aprobado por el profesor: C# sobre .NET 10, MonoGame DesktopGL, CoreWCF y SQL Server, sin tecnologías web.

**Drivers principales.** Que un cliente modificado no pueda hacer trampa (QAS-01) y que las contraseñas resistan intentos en serie (QAS-02); que una jugada llegue al rival en menos de 500 ms (QAS-03); que una desconexión o una falla de la base o del correo no deje partidas, cierres ni sanciones a medias (QAS-05 a QAS-07, QAS-12); y que todo funcione igual en los dos sistemas (QAS-08). De ahí salen siete ASR: que nada dependa de que el cliente sea honesto (ASR-01), que los cambios lleguen a tiempo a cada cliente (ASR-02), multiplataforma (ASR-03), cuentas y sesiones que resistan la adivinación y el robo de datos

(ASR-04), ninguna operación a medias ni repetida (ASR-05), reglas que se prueban sin infraestructura y servicios externos sustituibles (ASR-06) e internacionalización (ASR-07). El servidor como única autoridad no es un driver: es la respuesta que se elige en ADR-001 tras compararla con otras.

**Estructura general.** Cliente-servidor en capas (ADR-001): el cliente MonoGame (C-01) solo conoce los contratos (C-02); el servidor (C-03) expone CoreWCF, sus servicios (C-04) validan con un dominio sin dependencias (C-05), llevan en memoria la cola de emparejamiento y las salas, y guardan con EF Core (C-06) en SQL Server (C-07); el correo y el segundo factor salen por una pasarela (C-08). Las vistas V-01 y V-02 lo muestran.

**Decisiones esenciales.** CoreWCF dúplex sobre net.tcp con llamadas asíncronas (ADR-002); MonoGame DesktopGL (ADR-003); canal cifrado con TLS y servidor reconocido por la huella de su certificado (ADR-007, que sustituye al `SecurityMode.None` de ADR-004); EF Core y SQL Server 2022 con el esquema en scripts SQL (ADR-005); resultados codificados traducidos en el cliente (ADR-006).

**Riesgos abiertos.** La custodia del certificado del servidor y la distribución de su huella (R-01), la presentación del tablero (R-02), SQL Server bajo Rosetta (R-03), el servidor como punto único de falla (R-04), la latencia aún sin medir (R-05) y un esquema y un código que todavía reflejan el alcance anterior (R-09).

## 1.3 Glosario

| Término | Significado en este proyecto |
|---|---|
| Cliente | Aplicación de escritorio (MonoGame DesktopGL) que dibuja las pantallas y envía las acciones del jugador. No decide nada: solo muestra el estado que le da el servidor. |
| Servidor de partidas | Proceso .NET que expone los servicios CoreWCF, valida toda regla del juego y de la cuenta, y es el único que accede a la base de datos. |
| Contrato | Interfaz de servicio o de callback y sus tipos de datos (`Bastion.Contracts`), compartidos por cliente y servidor. |
| Callback | Llamada que el servidor hace al cliente por el canal dúplex para avisarle de un cambio (jugada del rival, mensaje de chat, fin de partida) sin que el cliente lo pida. |
| Dúplex net.tcp | Canal TCP binario de CoreWCF que permite llamadas en ambos sentidos sobre la misma conexión. En Bastión va cifrado con TLS (ADR-007). |
| Huella del certificado | Resumen SHA-256 del certificado TLS del servidor. El cliente solo se conecta si coincide con la que tiene configurada (ADR-007). |
| Token de sesión | Valor aleatorio que el servidor entrega al iniciar sesión y que acompaña cada llamada posterior. Solo se guarda su resumen SHA-256; vence a las 24 horas (CU-01 RN-07). |
| Partida | Juego entre dos o cuatro participantes en un *modo*. Termina por llegada a meta, reloj, rendición, tablas o abandono (CU-19 RN-01). |
| Participación | Registro de un jugador en una partida, con su elo de entrada y su resultado. Un jugador tiene como mucho una sin terminar (CU-09 RN-01, D-01). |
| Modo | Variante de juego: clásico 9×9 a dos jugadores con diez muros, cuatro jugadores 9×9 con cinco muros, rápida 7×7 con seis muros (CU-09 RN-02). |
| Muro | Pieza de dos casillas que se coloca en el surco entre casillas; nunca puede dejar a un jugador sin camino a su meta (CU-14 RN-05). |
| Elo | Puntuación de habilidad por modo que cambia al terminar una partida clasificatoria y ordena el ranking. |
| Partida clasificatoria | La que se crea por emparejamiento (CU-09). Es la única que cambia el elo y las estadísticas; las de sala privada son amistosas (CU-09 RN-09, CU-19 RN-03). |
| Sala privada | Sala de espera con un código de seis caracteres en la que el anfitrión fija los ajustes y de la que sale una partida amistosa. Se entra con el código o con la invitación de un amigo (CU-10, CU-11, CU-24, D-29). |
| Invitado | Cuenta sin correo ni contraseña, atada a un token guardado en el dispositivo. Solo entra a partidas con el código de una sala, nunca clasifica y no tiene amigos (CU-03, D-29). |
| Requisito deseable | Función analizada que queda fuera de esta versión, con identificador DES-xx; ningún caso del alcance depende de ella (D-25). |
| Strike | Falta que el sistema registra cuando un jugador reúne reportes pendientes de tres jugadores distintos. Cada strike tiene una sanción fija y se aplica sin moderadores (CU-28 RN-10 a RN-12, D-26). |
| Sanción | Restricción que el sistema aplica de forma automática con cada strike, de ámbito cuenta o chat, temporal o permanente (D-05, CU-28 RN-11). |
| Bitácora de accesos | Registro de solo inserción de los intentos de acceso y de los fallos del segundo factor (CU-01, CU-06 RN-09). Consultarla desde la aplicación es un requisito deseable (DES-22). |
| Resultado codificado | Valor de enumeración que el servidor devuelve en lugar de un texto; el cliente lo traduce con sus recursos `.resx`. |
| Usuario de mínimo privilegio | Inicio de sesión de SQL Server `BastionServerConnection`, sin permiso |

### 1.4 Uso de IA generativa

*Una fila por uso significativo. No anexen conversaciones.*

| Qué pidieron | Qué aceptaron y qué rechazaron | Cómo lo verificaron |
|---|---|---|
| Apoyo en investigación: comparar opciones de interfaz (MonoGame, Godot, Unity, Stride, Avalonia), de comunicación (CoreWCF frente a WCF de .NET Framework) y de base de datos que funcionaran en macOS arm64 y en Windows. | Se aceptaron como punto de partida los resúmenes de ventajas y limitaciones; se rechazó toda opción prohibida por el curso o que no corriera en ambos sistemas. La elección final de cada tecnología la tomó el equipo y está en sus ADR. | Se consultó la documentación oficial de cada tecnología y se compiló y ejecutó un proyecto mínimo en las dos máquinas del equipo antes de adoptarla. |
| Completación de código: sugerencias al escribir servicios, repositorios, pantallas y pruebas. | Se aceptaron las sugerencias que cumplían el estándar de codificación y las reglas de dependencia; se rechazaron las que ponían reglas en el cliente o mezclaban capas. | Revisión del código por ambos integrantes, compilación con `EnforceCodeStyleInBuild` y pruebas unitarias con xUnit. |
| Elaboración del documento: redacción y formato en LaTeX de esta plantilla a partir de los casos de uso y de las decisiones del equipo. | Se aceptó la redacción que reflejaba lo decidido; se corrigió o eliminó lo que no correspondía a los casos de uso o al código. | Los autores revisaron y validaron cada sección contra los casos de uso, el modelo de datos y el repositorio del código. |

## 2. Drivers Primera entrega

*Expliquen qué obliga a la arquitectura a tomar cierta forma. Un ASR puede venir de un escenario de calidad, pero también de una función, una restricción, un objetivo de negocio o una condición del equipo.*

### 2.1 Contexto y frontera

*En un párrafo: qué está dentro del sistema, qué queda fuera y con qué sistemas externos se comunica. El diagrama va en la sección 6 como V-01; aquí sólo refiéranlo.*

Dentro del sistema están el **cliente de escritorio** (pantallas, entrada del jugador, idiomas en-US y es-MX), el **servidor de partidas** (cuentas y segundo factor, sesiones, emparejamiento, salas privadas, partidas en curso, chat, amigos e invitaciones, ranking e historial, reportes con sus strikes y bitácora de accesos) y la **base de datos** SQL Server. Los actores humanos son dos: el Jugador, con cuenta registrada, y el Invitado, que solo entra a partidas con el código de una sala (lista de casos de uso, D-29). No hay moderadores ni administradores: las sanciones las aplica el sistema por strikes (D-26). Fuera del sistema quedan el **servicio de correo**, que el servidor usa para verificar la cuenta al registrarse y para avisar de cada cambio de seguridad: contraseña, activación y desactivación del segundo factor (CU-02, CU-04 RN-05, CU-05 RN-06, CU-06 RN-06), y el **proveedor del segundo factor**, que entrega o valida el código de la autenticación de doble factor, ya sea por correo electrónico o con una aplicación autenticadora TOTP como Google Authenticator (CU-01 FA-10, RN-08; CU-05; CU-06); los dos pueden fallar, tardar o cambiar de proveedor (CU-01 EX-06, CU-02 EX-06). El sistema no envía invitaciones por correo (D-29), el correo de una cuenta no se cambia (D-28) y no hay economía ni pagos, ni navegador, ni servicios web. El diagrama de contexto es la vista V-01 (sección 6).

### 2.2 Interesados y preocupaciones

*Incluyan sólo a quienes tienen una preocupación que puede cambiar la arquitectura. Una preocupación es lo que le importa a alguien, no una función del sistema.*

| Interesado | Qué le preocupa |
|---|---|
| STK-01 · Jugador (8 años o más) | **Interés:** Reglas fáciles de aprender, controles accesibles, partidas fluidas y profundidad estratégica suficiente.<br>CRN-01: Entender al momento por qué una jugada no es válida, sin leer un párrafo.<br>CRN-02: Que el juego responda en menos de un segundo al colocar una barrera.<br>CRN-03: Que el juego siga interesando sin que aprender cueste más de unos minutos.<br>CRN-06: Saber qué pasa si se corta la conexión a mitad de partida: volver, perder o dejar esperando al rival.<br>CRN-18: No ser sancionado por reportes de mala fe sin que nadie revise lo ocurrido. |
| STK-02 · Padre o tutor | **Interés:** Contenido seguro, protección del menor y control sobre lo que hace su hijo.<br>CRN-04: Que en el chat no haya insultos ni desconocidos que hablen con el menor.<br>CRN-05: Entregar solo los datos imprescindibles del menor y saber qué se guarda y para qué. |
| STK-03 · Profesor o cliente | **Interés:** Cumplimiento de objetivos y entrega puntual.<br>CRN-07: Saber si lo comprometido cabe en las 14 semanas y qué se recorta primero.<br>CRN-08: Ver cada semana algo que funcione y se pueda probar. |
| STK-04 · Director del proyecto | **Interés:** Cumplimiento de alcance, presupuesto y calendario.<br>CRN-07: Saber si lo comprometido cabe en las 14 semanas y qué se recorta primero.<br>CRN-14: Avanzar sin que la decisión sobre IA, conexión LAN o nombre del juego obligue a rehacer trabajo. |
| STK-05 · Arquitecto de software | **Interés:** Estructura coherente, modificabilidad y decisiones justificadas.<br>CRN-09: Poder justificar cada decisión estructural y señalar dónde y por qué se tomó.<br>CRN-10: Decidir quién manda sobre el estado de la partida, el cliente o el servidor.<br>CRN-11: Definir el protocolo sobre TCP sin WebSockets: delimitación de mensajes, serialización y versiones distintas de cliente y servidor.<br>CRN-12: Que cliente y servidor apliquen exactamente las mismas reglas.<br>CRN-20: Añadir LAN sin rehacer el núcleo, con el extremo de red configurable desde fuera.<br>CRN-26: Dejar el segundo factor detrás de una interfaz para cambiar de mecanismo sin tocar el inicio de sesión. |
| STK-06 · Diseñador del videojuego | **Interés:** Mecánicas originales y equilibradas.<br>CRN-03: Que el juego siga interesando sin que aprender cueste más de unos minutos.<br>CRN-13: Cambiar el tamaño del tablero y el número de barreras sin recompilar. |
| STK-07 · Programadores | **Interés:** Requisitos claros, recursos y tiempo suficientes.<br>CRN-11: Definir el protocolo sobre TCP sin WebSockets: delimitación de mensajes, serialización y versiones distintas de cliente y servidor.<br>CRN-14: Avanzar sin que la decisión sobre IA, conexión LAN o nombre del juego obligue a rehacer trabajo. |
| STK-08 · Equipo de arte, interfaz y sonido | **Interés:** Identidad visual y sonora propia e interfaz fácil de usar.<br>CRN-15: Que la interfaz no se descuadre al traducir textos de distinta longitud. |

| Interesado | Qué le preocupa |
|---|---|
| STK-09 · Diseñador de experiencia de usuario | **Interés:** Facilidad de aprendizaje y buena experiencia de uso.<br>CRN-01: Entender al momento por qué una jugada no es válida, sin leer un párrafo.<br>CRN-15: Que la interfaz no se descuadre al traducir textos de distinta longitud. |
| STK-10 · Equipo de pruebas (QA) | **Interés:** Estabilidad, ausencia de errores y equilibrio.<br>CRN-12: Que cliente y servidor apliquen exactamente las mismas reglas.<br>CRN-17: Ejecutar y repetir partidas completas sin pasar por la interfaz.<br>CRN-23: Que un cliente modificado no pueda hacer jugadas ilegales ni ver información oculta del rival. |
| STK-11 · Equipo de localización | **Interés:** Traducciones claras y culturalmente apropiadas.<br>CRN-15: Que la interfaz no se descuadre al traducir textos de distinta longitud.<br>CRN-16: Traducir textos, fechas, números y formatos sin tocar el código. |
| STK-13 · Administrador de base de datos | **Interés:** Integridad, disponibilidad y recuperación de los datos.<br>CRN-19: Recuperar cuentas, progreso y partidas tras una caída, y definir qué información es aceptable perder.<br>CRN-21: Saber qué hace el sistema si el correo o un servicio externo deja de responder, sin quedarse colgado. |
| STK-14 · Equipo de operaciones y despliegue | **Interés:** Que el sistema sea desplegable y esté disponible.<br>CRN-06: Saber qué pasa si se corta la conexión a mitad de partida: volver, perder o dejar esperando al rival.<br>CRN-20: Añadir LAN sin rehacer el núcleo, con el extremo de red configurable desde fuera.<br>CRN-21: Saber qué hace el sistema si el correo o un servicio externo deja de responder, sin quedarse colgado. |
| STK-15 · Especialista en ciberseguridad | **Interés:** Seguridad de las cuentas y prevención de trampas.<br>CRN-10: Decidir quién manda sobre el estado de la partida, el cliente o el servidor.<br>CRN-18: No ser sancionado por reportes de mala fe sin que nadie revise lo ocurrido.<br>CRN-22: Que el tráfico TCP en claro permita leer credenciales y secuestrar sesiones.<br>CRN-23: Que un cliente modificado no pueda hacer jugadas ilegales ni ver información oculta del rival.<br>CRN-26: Dejar el segundo factor detrás de una interfaz para cambiar de mecanismo sin tocar el inicio de sesión. |
| STK-16 · Auditor del proyecto | **Interés:** Trazabilidad y evidencia verificable cada semana.<br>CRN-08: Ver cada semana algo que funcione y se pueda probar.<br>CRN-09: Poder justificar cada decisión estructural y señalar dónde y por qué se tomó. |
| STK-17 · Asesor legal y de propiedad intelectual | **Interés:** Uso legal de nombres, diseños, reglas y recursos.<br>CRN-04: Que en el chat no haya insultos ni desconocidos que hablen con el menor.<br>CRN-05: Entregar solo los datos imprescindibles del menor y saber qué se guarda y para qué.<br>CRN-24: Saber hasta dónde puede parecerse el juego a Quoridor sin infringir derechos de terceros. |
| STK-18 · Gigamic y Mirko Marchesi | **Interés:** Protección de su marca y propiedad intelectual.<br>CRN-24: Saber hasta dónde puede parecerse el juego a Quoridor sin infringir derechos de terceros. |

| Interesado | Qué le preocupa |
|---|---|
| STK-19 · Plataformas de distribución (restricción legal) | **Interés:** Cumplimiento de normas técnicas, legales y de contenido.<br>CRN-04: Que en el chat no haya insultos ni desconocidos que hablen con el menor.<br>CRN-25: Exigir clasificación por edad, empaquetado conforme y una política de chat que cumpla la regulación de cada región. |

STK-12 (equipo de soporte y moderación) se retiró en la v0.2: esta versión no tiene moderadores (D-26). Sus preocupaciones, CRN-04 y CRN-18, siguen presentes en STK-01, STK-02 y STK-15.

## 2.3 Funciones clave

*Usen los casos de uso o las historias de usuario que ya tienen, con sus identificadores. Señalen cuáles obligan a tomar decisiones arquitectónicas. Las demás pueden seguir la estructura general del sistema y documentarse sólo cuando sea necesario.*

| Función (CU o HU) | ¿Arquitectónicamente significativa? | Por qué |
|---|---|---|
| CU-01 · Iniciar sesión | Sí | Las credenciales viajan por la red y alguien puede intentar adivinarlas o interceptarlas; no pueden transmitirse ni guardarse en claro (RN-02); la respuesta no debe revelar si la cuenta existe (RN-01) y los intentos fallidos repetidos deben frenarse (RN-03); la sesión dura 24 horas y una sanción debe cortarla (RN-07). Autenticación de doble factor (CON-08): si la cuenta la tiene activada, no se entra hasta comprobar un código de un solo uso con vigencia de cinco minutos y tres intentos (FA-10, RN-09). El código lo entrega un servicio externo, por correo electrónico o con una aplicación autenticadora TOTP como Google Authenticator, que puede fallar o tardar (EX-06), y el mecanismo aún no se elige y puede cambiar (RN-08, Q-03). |
| CU-02, CU-04, CU-05, CU-06 · Registro con verificación del correo, cambio de contraseña y segundo factor | Sí | Dependen de un servicio de correo externo que puede fallar o tardar: el envío queda fuera de la transacción y su fallo no deshace la cuenta ni el cambio (CU-02 RN-07, EX-06; CU-04 RN-05). Los códigos caducan, son de un solo uso y solo se guarda su resumen (CU-02 RN-11, CU-05 RN-03). Una contraseña actual incorrecta cuenta para el bloqueo escalonado (CU-04 RN-07), y desactivar el segundo factor exige además un código vigente (CU-06 RN-01). |
| CU-03 · Jugar como invitado | Sí | Una cuenta sin credenciales se identifica solo por un token guardado en el dispositivo (RN-03, RN-04); las altas por dirección de red están acotadas (RN-07), y lo que un invitado no puede hacer (clasificar, tener amigos, invitar, reportar) debe impedirlo el servidor en cada operación, no el cliente al ocultar la opción (RN-05, D-29). |

| Función (CU o HU) | ¿Arquitectónicamente significativa? | Por qué |
|---|---|---|
| CU-09 · Buscar partida clasificatoria | Sí | Muchos jugadores esperan a la vez en una cola que vive en memoria (D-24); la partida no puede quedar creada a medias, con unos participantes sí y otros no (RN-08), y un jugador no puede estar en dos partidas sin terminar (RN-01, D-01). Es la única fuente de partidas que mueven el elo (RN-09). |
| CU-10, CU-11, CU-12 · Salas privadas | Sí | Las plazas y los expulsados de cada sala viven en la memoria del servidor (D-24, D-27); dos jugadores no deben ocupar a la vez la última plaza (CU-11 RN-02); los códigos no deben poder adivinarse probando (CU-11 RN-04); solo el anfitrión expulsa, y el servidor lo comprueba en cada solicitud (CU-12 RN-01, RN-03). |
| CU-13 · Mover el peón y CU-14 · Colocar un muro | Sí | Un jugador puede intentar una jugada ilegal o fuera de turno, incluso con un cliente modificado, y ninguna debe alterar la partida (CU-13 RN-01, RN-07; CU-14 RN-08); una jugada repetida por la red no debe aplicarse dos veces (CU-13 RN-06, RN-13); un muro nunca puede cerrar el camino de ningún jugador, lo que obliga a recorrer el tablero en cada colocación (CU-14 RN-05); el rival debe ver la jugada casi al instante. |
| CU-19 · Finalizar la partida | Sí | El cierre de la partida, los resultados, el elo, las estadísticas y, si viene de una sala, el cierre de la sala cambian juntos o no cambia ninguno (RN-10); una falla no debe perder el cierre de la partida (RN-11). |
| CU-18 · Abandonar y CU-20 · Reconectar | Sí | Fallas de red: la desconexión no detiene el reloj, el rival puede reclamar a los 60 segundos (CU-18 RN-06) y el jugador que vuelve debe ver el tablero exactamente como quedó (CU-20 RN-02, RN-06); una partida admite una sola conexión por participante (CU-20 RN-04) y la caída del propio servidor no se descuenta del reloj (CU-20 RN-05). |
| CU-21 · Enviar un mensaje de chat | Sí | Ningún mensaje con palabras prohibidas debe llegar a otro jugador, aunque el emisor modifique su cliente; si el filtro no puede aplicarse, no se entrega (RN-01, RN-08); el ritmo de envío está acotado (RN-04); cada canal, de sala o de partida, lo leen solo sus miembros (RN-10), y los emotes se reenvían sin guardarse (RN-13). |
| CU-24 · Invitar a un amigo a jugar | Sí | El servidor debe saber qué amigos están conectados y libres, y hacer llegar la invitación a un cliente que no comparte sala ni partida con quien invita; la invitación caduca a los 60 segundos (RN-03, RN-05). |

| Función (CU o HU) | ¿Arquitectónicamente significativa? | Por qué |
|---|---|---|
| CU-28 · Reportar a un jugador | Sí | Moderación sin moderadores (CON-09, D-26): el strike se aplica en la misma transacción que crea el tercer reporte de jugadores distintos (RN-10), y una sanción de cuenta surte efecto de inmediato, aunque el jugador tenga sesiones abiertas o esté en una sala o una partida (RN-13). Para que nadie sea sancionado por reportes de mala fe (CRN-18), el número de reportes por jugador está acotado y los invitados no reportan (RN-06, RN-07). |
| CU-07, CU-08, CU-15 a CU-17, CU-22, CU-23, CU-25 a CU-27 | No | No plantean necesidades distintas de las de los casos anteriores (sesión, jugadas válidas, cambios completos o ninguno) y siguen la estructura general que resulte de las decisiones de la sección 3. |

## 2.4 Restricciones, supuestos y condiciones del negocio

*Una restricción viene impuesta: el equipo no la decide. Un supuesto es algo que creen cierto pero no han comprobado. Si una fila habla de tiempos de respuesta o de carga, probablemente es un escenario (2.5).*

| Tipo | Enunciado | Qué pasa si cambia o resulta falso |
|---|---|---|
| Restricción | El curso pide comunicación dúplex sobre net.tcp con llamadas asíncronas y prohíbe el stack web, WebSockets, SignalR, gRPC y REST (CON-02, CON-03). | Se ampliarían las opciones de comunicación y habría que volver a comparar las alternativas de ADR-002. |
| Restricción | El lenguaje es C# sobre .NET; el sistema es una aplicación de escritorio cliente-servidor (CON-01). | Cambiar de lenguaje o de plataforma invalidaría todas las decisiones tecnológicas. |
| Restricción | El sistema debe compilar y ejecutarse en macOS arm64 y en Windows, porque cada integrante trabaja en uno de ellos. | Si ambos usaran Windows, volverían a ser viables tecnologías solo de Windows, como WCF de .NET Framework o MonoGame WindowsDX. |
| Restricción | La base de datos es SQL Server y los textos deben guardarse en Unicode (CON-04, CON-05). | Otro motor obligaría a rehacer el modelo de datos ya entregado en la actividad A1. |
| Restricción | El inicio de sesión debe ofrecer autenticación de doble factor (CON-08); el mecanismo (correo electrónico, aplicación autenticadora TOTP u otro) queda a criterio del equipo (Q-03). | Obliga a depender de un servicio externo al iniciar sesión y a proteger secretos y códigos; si se quitara, el inicio de sesión volvería a un solo paso y sobrarían CU-05 y CU-06. |
| Restricción | La moderación es simple, por reportes, sin moderadores (CON-09, D-26). | Con moderadores habría un rol con permisos propios, una cola de reportes y apelaciones (DES-17 a DES-21): más contratos y autorización por rol, no una estructura distinta. |
| Restricción | El juego se ofrece en inglés y en español (CON-11). | Sin el segundo idioma bastaría un solo archivo de recursos, pero la decisión de traducir en el cliente (ADR-006) seguiría siendo válida. |

| Tipo | Enunciado | Qué pasa si cambia o resulta falso |
|---|---|---|
| Restricción | El cliente fijó el alcance en 28 casos de uso; los 23 restantes quedan como requisitos deseables (D-25). | Recuperar deseables con exigencias propias, como la difusión a espectadores, la IA o la economía de monedas, traería escenarios y decisiones que esta versión no contempla. |
| Supuesto | El juego se usa en una red local o de aula, con decenas de jugadores conectados a la vez y no miles. | Con miles de jugadores la carga cambiaría por órdenes de magnitud y habría que replantear los escenarios de desempeño (QAS-03) y el servidor único (R-04). |
| Supuesto | El atacante esperado es un jugador con un cliente modificado o alguien en la misma red que observa el tráfico; nadie ajeno al equipo tiene acceso a la máquina del servidor ni a su certificado. | Si alguien obtuviera el certificado y su contraseña podría hacerse pasar por el servidor; habría que cambiar el certificado y la huella de cada cliente (ADR-007, R-01). |
| Objetivo de negocio | Que las partidas clasificatorias sean justas: el ranking solo vale si nadie puede hacer trampa ni perder elo por una falla del sistema, y por eso solo clasifican las partidas del emparejamiento, para que dos amigos no se regalen puntos (CU-10 RN-03). | Si el ranking no importara, bajaría la prioridad de QAS-01 y QAS-06, aunque las reglas del juego seguirían teniendo que cumplirse. |
| Objetivo de negocio | Que el juego sea apto para jugadores desde ocho años: ningún mensaje con palabras prohibidas llega a otro jugador y la mala conducta se sanciona sin depender de un moderador (CU-02 RN-05, CU-21, CU-28). | Sin menores entre los jugadores, el filtro y los strikes podrían relajarse, pero seguirían siendo parte de la moderación que exige CON-09. |
| Condición organizacional | Equipo de dos personas, una en macOS arm64 y otra en Windows, durante el periodo agosto 2026 – enero 2027, en catorce semanas con incrementos semanales auditables y entregas parciales evaluadas. | Con menos tiempo se recortan primero los casos *Should* (CU-15 a CU-17, CU-25 y CU-27), no la estructura; los deseables ya quedaron fuera. |
| Condición organizacional | El repositorio remoto se alojará en GitLab o Bitbucket, según lo que pida el curso (decisión abierta, R-06). | No afecta la arquitectura; solo el flujo de trabajo. |

## 2.5 Escenarios de calidad y árbol de utilidad

*Entre 8 y 12 escenarios medibles; copien la ficha para cada uno. La respuesta dice qué hace el sistema y la medida dice cómo se comprueba. No escriban la solución dentro del escenario.*

| QAS-01 · Seguridad · Jugada ilegal desde un cliente modificado | |
|---|---|
| **Fuente** | Un jugador con un cliente modificado o con una herramienta propia que imita al cliente |
| **Estímulo** | Envía una jugada ilegal: mover el peón dos casillas sin salto, colocar un muro que encierra al rival, jugar fuera de turno o repetir el número de orden de una jugada ya aplicada |
| **Entorno** | Partida clasificatoria en curso, operación normal |
| **Artefacto** | Las jugadas de una partida en curso (CU-13, CU-14) |
| **Respuesta** | El sistema rechaza la jugada, no cambia el tablero ni el turno, avisa al jugador por qué no es válida y deja registro del intento |
| **Medida** | El 100 % de los casos de una batería de al menos 30 jugadas ilegales (una por regla de CU-13 y CU-14) se rechaza y el estado guardado de la partida queda idéntico; se comprueba enviando la batería desde un cliente de prueba |
| **Prioridad** | (A, A) |
| **Activo que se protege** | El estado de la partida y, por él, el elo y las estadísticas que otorga |
| **Actor de amenaza** | Jugador autenticado que controla su propio cliente |
| **Límite de confianza** | Entre el equipo del jugador y el resto del sistema: lo que llega desde el cliente no es confiable |

| QAS-02 · Seguridad · Adivinación de contraseñas | |
|---|---|
| **Fuente** | Un atacante que conoce el nickname o el correo de una víctima |
| **Estímulo** | Prueba contraseñas en serie contra el inicio de sesión |
| **Entorno** | Operación normal; el atacante puede conectarse al sistema desde la misma red |
| **Artefacto** | Inicio de sesión y datos de las cuentas |
| **Respuesta** | Responde siempre con el mismo resultado genérico, bloquea la cuenta de forma escalonada y registra cada intento en la bitácora de accesos sin la contraseña recibida |
| **Medida** | Al tercer fallo consecutivo la cuenta queda bloqueada 5 minutos, al sexto 30 y desde el noveno 60 (CU-01 RN-03); en 100 intentos, la diferencia entre el tiempo medio de respuesta con cuenta inexistente y con contraseña incorrecta es menor a 50 ms; ninguna fila de la base ni línea de bitácora contiene una contraseña en claro. Se comprueba con un cliente de prueba que lanza los intentos y con una revisión de la base y de las bitácoras |
| **Prioridad** | (A, M) |
| **Activo que se protege** | Las credenciales de la cuenta y todo lo que depende de ella: elo, amigos, historial y la posibilidad de reportar |
| **Actor de amenaza** | Persona sin la contraseña de la víctima, con un cliente propio y acceso a la red del servidor |
| **Límite de confianza** | Entre una persona aún no autenticada y el inicio de sesión; las credenciales guardadas no deben poder leerse aunque alguien obtenga una copia de los datos |

| QAS-03 · Desempeño · Jugada vista por el rival | |
|---|---|
| **Fuente** | Un jugador |
| **Estímulo** | Confirma una jugada válida en su turno |
| **Entorno** | Red local del aula, con 20 partidas simultáneas en el servidor |
| **Artefacto** | La partida en curso y el cliente del rival |
| **Respuesta** | El sistema acepta la jugada, la guarda y la muestra en el tablero de todos los participantes |
| **Medida** | El percentil 95 del tiempo entre que el jugador confirma la jugada y el cliente del rival la recibe es menor a 500 ms; se mide con marcas de tiempo en las bitácoras de ambos clientes en una prueba de carga con clientes simulados |
| **Prioridad** | (A, A) |

| QAS-04 · Desempeño · Inicio de sesión | |
|---|---|
| **Fuente** | Un jugador registrado |
| **Estímulo** | Inicia sesión con credenciales correctas |
| **Entorno** | Operación normal, sin segundo factor activado |
| **Artefacto** | Inicio de sesión |
| **Respuesta** | Comprueba las credenciales y da acceso al menú principal |
| **Medida** | Menos de 1 segundo desde que el jugador pulsa «Iniciar sesión» hasta que ve el menú, sin contar servicios externos (CU-01 RN-11), en el 95 % de 50 intentos; se mide con cronómetro en el cliente |
| **Prioridad** | (M, M) |

| QAS-05 · Disponibilidad · Desconexión de un jugador | |
|---|---|
| **Fuente** | La red del jugador o el cierre inesperado de su cliente |
| **Estímulo** | El jugador pierde la conexión a mitad de una partida |
| **Entorno** | Partida en curso, el servidor sigue funcionando |
| **Artefacto** | Partida en curso y cliente del jugador |
| **Respuesta** | La partida continúa con el reloj corriendo; el rival puede reclamar la victoria a los 60 segundos; si el jugador vuelve, ve el tablero como quedó y sigue jugando (CU-18, CU-20) |
| **Medida** | Ninguna jugada guardada se pierde; tras reconectar, el tablero que ve el jugador coincide con el de su rival en menos de 2 segundos; se comprueba desconectando el cable o cerrando el cliente en 10 partidas de prueba |
| **Prioridad** | (A, A) |

| **QAS-06 · Confiabilidad · Falla de la base de datos al cerrar una partida** | |
|---|---|
| **Fuente** | La base de datos |
| **Estímulo** | Deja de responder mientras se escriben los resultados de una partida terminada |
| **Entorno** | Cierre de una partida clasificatoria (CU-19) |
| **Artefacto** | Cierre de la partida: resultados, elo, estadísticas y participaciones |
| **Respuesta** | No queda escrito ningún resultado parcial; la partida se marca pendiente de cierre y se reintenta (CU-19 RN-10, RN-11) |
| **Medida** | Cero partidas con el elo aplicado y las estadísticas o las participaciones sin cerrar (o al revés) en 20 fallas provocadas; la partida se cierra en menos de 1 minuto después de que la base vuelve; se comprueba deteniendo la base de datos durante el cierre |
| **Prioridad** | (A, M) |

| **QAS-07 · Integridad · Sanción automática por reportes simultáneos** | |
|---|---|
| **Fuente** | Varios jugadores distintos |
| **Estímulo** | Envían al mismo tiempo reportes contra un jugador que ya tiene dos denunciantes pendientes, de modo que cualquiera de ellos completa el tercero |
| **Entorno** | Operación normal; el reportado tiene una sesión abierta y está en una partida en curso |
| **Artefacto** | Reportes, strikes y sanciones de la cuenta (CU-28) |
| **Respuesta** | Se aplica exactamente un strike con su sanción, todos los reportes pendientes pasan a atendidos y, si la sanción es de cuenta, el reportado pierde sus sesiones y sale de la partida como si la abandonara (CU-28 RN-10, RN-13) |
| **Medida** | En 50 pruebas con tres a cinco reportes simultáneos, cada reportado recibe un solo strike y no queda ningún reporte pendiente que ya debió atenderse; con una sanción de cuenta, el cliente del sancionado pierde el acceso en menos de 2 segundos; se comprueba con clientes de prueba y una consulta SQL sobre la base |
| **Prioridad** | (A, M) |

| **QAS-08 · Portabilidad · Dos sistemas operativos** | |
|---|---|
| **Fuente** | Un integrante del equipo |
| **Estímulo** | Clona el repositorio en macOS arm64 o en Windows, compila y ejecuta |
| **Entorno** | Desarrollo, con las herramientas que el equipo usa en cada sistema |
| **Artefacto** | Toda la solución |
| **Respuesta** | Compila, pasa las pruebas y el cliente de un sistema juega una partida completa contra un servidor en el otro |
| **Medida** | Cero cambios de código o de configuración del repositorio entre sistemas; se comprueba antes de cada entrega con una partida cruzada Mac–Windows en ambos sentidos |
| **Prioridad** | (A, M) |

| QAS-09 · Modificabilidad · Cambio del proveedor de correo | |
|---|---|
| **Fuente** | El equipo |
| **Estímulo** | Necesita cambiar el servicio de correo o el mecanismo del segundo factor (CU-01 RN-08) |
| **Entorno** | Desarrollo |
| **Artefacto** | Envío de correos y verificación del segundo factor |
| **Respuesta** | Se reemplaza la implementación sin tocar los flujos de los casos de uso |
| **Medida** | El cambio no toca los flujos de los casos de uso ni el cliente y se completa en menos de un día de trabajo; se comprueba revisando el diff del cambio |
| **Prioridad** | (B, B) |

| QAS-10 · Capacidad de prueba · Reglas sin infraestructura | |
|---|---|
| **Fuente** | Un integrante del equipo |
| **Estímulo** | Ejecuta las pruebas de las reglas del juego y de los servicios |
| **Entorno** | Desarrollo, sin base de datos ni servidor en ejecución |
| **Artefacto** | Reglas del juego y de los casos de uso |
| **Respuesta** | Las pruebas se ejecutan sin base de datos ni red y reportan el resultado de cada regla |
| **Medida** | El 100 % de las reglas de negocio de CU-13 y CU-14 tiene al menos una prueba; la suite unitaria completa termina en menos de 30 segundos en cualquiera de las dos máquinas |
| **Prioridad** | (M, B) |

| QAS-11 · Usabilidad · Cambio de idioma | |
|---|---|
| **Fuente** | Un jugador |
| **Estímulo** | Cambia el idioma de en-US a es-MX |
| **Entorno** | Operación normal |
| **Artefacto** | Pantallas y mensajes del juego |
| **Respuesta** | Todas las pantallas y todos los mensajes de error se muestran en el idioma elegido; el chat no se traduce (CU-21 RN-09) |
| **Medida** | Cero textos en otro idioma al recorrer todas las pantallas y provocar cada mensaje de error en es-MX; agregar un tercer idioma no requiere cambiar código; se comprueba con el recorrido y con una prueba automática que verifica que no falte ninguna traducción |
| **Prioridad** | (M, B) |

| QAS-12 · Disponibilidad · Servicio de correo caído | |
|---|---|
| **Fuente** | El servicio de correo externo |
| **Estímulo** | Deja de responder mientras un jugador se registra o cambia su contraseña |
| **Entorno** | Operación normal, con otros jugadores conectados y jugando |
| **Artefacto** | Registro (CU-02) y cambio de contraseña (CU-04) |
| **Respuesta** | La cuenta o el cambio se guardan aunque el correo no salga; el jugador sabe que el código no se envió y puede pedirlo de nuevo; el resto del sistema sigue atendiendo (CU-02 RN-07, EX-06; CU-04 RN-05) |
| **Medida** | Con el correo simulado sin respuesta, el 100 % de 20 registros crea la cuenta y responde en menos de 10 segundos; mientras tanto, el inicio de sesión de otros jugadores sigue cumpliendo QAS-04; se comprueba con una implementación de prueba de C-08 que nunca responde |
| **Prioridad** | (M, M) |

**Escenario de seguridad.** Al menos uno. Agreguen a su ficha tres filas: activo que se protege, actor de amenaza y límite de confianza.

*Árbol de utilidad: insértenlo aquí o usen una tabla equivalente. Prioricen por importancia y dificultad.*

| Utilidad de Bastión | | | |
|---|---|---|---|
| **Atributo** | **Refinamiento** | **Escenario** | **(Imp., Dif.)** |
| **Seguridad** | Integridad de la partida frente a un cliente modificado | QAS-01 | (A, A) |
| | Protección de las credenciales frente a la adivinación | QAS-02 | (A, M) |
| **Desempeño** | Latencia de la jugada hacia el rival | QAS-03 | (A, A) |
| | Tiempo de inicio de sesión | QAS-04 | (M, M) |
| **Disponibilidad** | Continuidad de la partida ante desconexiones | QAS-05 | (A, A) |
| | Registro y cambios de cuenta con el correo caído | QAS-12 | (M, M) |
| **Confiabilidad e integridad** | Atomicidad del cierre de la partida | QAS-06 | (A, M) |
| | Un solo strike por umbral de reportes, con efecto inmediato | QAS-07 | (A, M) |
| **Portabilidad** | Mismo código en macOS arm64 y Windows | QAS-08 | (A, M) |
| **Mantenibilidad** | Sustituir servicios externos | QAS-09 | (B, B) |
| | Probar reglas sin infraestructura | QAS-10 | (M, B) |
| **Usabilidad** | Idiomas en-US y es-MX | QAS-11 | (M, B) |

A = alta, M = media, B = baja. Los escenarios (A, A) —QAS-01, QAS-03 y QAS-05— son los que más condicionan la arquitectura.

## 2.6 ASR prioritarios

*Entre 5 y 7. Si todos vienen de escenarios, expliquen en una línea por qué ninguna función, restricción, objetivo o condición del equipo obliga a estructurar el sistema.*

| ASR | Qué debe cumplir la arquitectura | Procedencia | Viene de |
|---|---|---|---|
| ASR-01 | Ninguna regla del juego, de la cuenta, de la sala ni de la moderación puede depender de que el cliente sea honesto: un cliente modificado no debe poder alterar una partida, saltarse lo que su cuenta no permite (invitado, sanción), actuar como anfitrión sin serlo ni evitar el filtro del chat. | Escenario, Función | QAS-01, CU-13, CU-14, CU-12 RN-01, CU-21 RN-01, CU-03 RN-05, D-29 |
| ASR-02 | Los cambios de una partida, de una sala, del chat y del emparejamiento, y las invitaciones de los amigos, deben llegar a cada cliente interesado en menos de 500 ms, con comunicación asíncrona y sin tecnologías prohibidas por el curso. | Escenario, Restricción | QAS-03, CU-09, CU-10, CU-21, CU-24, restricción de tecnologías del curso (CON-02, CON-03) |
| ASR-03 | Cliente y servidor deben compilarse y ejecutarse sin cambios en macOS arm64 y en Windows. | Restricción, Condición organizacional | QAS-08, restricción multiplataforma, equipo Mac–Windows |
| ASR-04 | Las cuentas y las sesiones deben resistir la adivinación de contraseñas y el robo de datos: ninguna contraseña, código del segundo factor ni token de sesión puede recuperarse a partir de lo que se guarda, se registra o viaja por la red; el inicio de sesión ofrece doble factor; una cuenta sancionada pierde el acceso de inmediato y la bitácora de accesos no puede alterarse. | Escenario, Supuesto, Restricción | QAS-02, QAS-07, CU-01 (FA-10, RN-02, RN-08, RN-09), CU-05, CU-06, CU-28 RN-13, CON-08, supuesto sobre el atacante (sección 2.4) |
| ASR-05 | Ninguna falla del cliente, de la red, de la base de datos o de un servicio externo puede dejar a medias una partida, su cierre, una sala, una sanción o una cuenta, ni aplicar dos veces la misma operación. | Escenario, Función | QAS-05, QAS-06, QAS-07, QAS-12, CU-09 RN-08, CU-11 RN-02, CU-19, CU-20, CU-28 RN-10 |
| ASR-06 | Las reglas del juego y de los casos de uso deben poder probarse sin base de datos ni red, y los servicios externos (correo, segundo factor) deben poder cambiarse sin tocar los flujos. | Condición organizacional, Escenario, Función | QAS-09, QAS-10, CU-01 RN-08, equipo de dos personas |
| ASR-07 | Todo texto visible, incluidos los errores, debe mostrarse en el idioma elegido (en-US o es-MX); agregar un idioma no debe requerir cambiar código y los textos de los jugadores | Restricción, Función, Escenario | QAS-11, CON-11, CU-01 RN-12, CU-21 RN-09, D-21 |

# 3. Decisiones

**Primera entrega · 4 a 6 ADR**

*Una ficha por decisión esencial. Cada decisión compara al menos dos alternativas reales. Una decisión que cambia no se borra: se marca como sustituida y la nueva la menciona.*

## 3.1 Índice de decisiones

| ADR | Decisión | Estado | ASR que atiende |
|---|---|---|---|
| ADR-001 | Organizar el sistema en capas cliente-servidor con el servidor como única autoridad | Aceptado | ASR-01, ASR-05, ASR-06 |
| ADR-002 | Comunicar cliente y servidor con CoreWCF dúplex sobre net.tcp y llamadas asíncronas | Aceptado | ASR-02, ASR-03 |
| ADR-003 | Construir el cliente con MonoGame DesktopGL | Aceptado | ASR-03 |
| ADR-004 | Usar `SecurityMode.None` y concentrar la seguridad en el servidor | Sustituido por ADR-007 | ASR-04, ASR-03 |
| ADR-005 | Persistir con EF Core y SQL Server 2022, con el esquema en scripts SQL | Aceptado | ASR-04, ASR-05, ASR-03 |
| ADR-006 | Traducir en el cliente los resultados codificados del servidor | Aceptado | ASR-07 |
| ADR-007 | Cifrar el canal net.tcp con TLS y reconocer al servidor por la huella de su certificado | Aceptado | ASR-04, ASR-03 |

## 3.2 Fichas de decisión

| **ADR-001 · Organizar el sistema en capas cliente-servidor con el servidor como única autoridad** | |
|---|---|
| **Estado** | Aceptado |
| **Contexto** | Un cliente modificado no debe poder hacer trampa (QAS-01), una sanción automática debe aplicarse una sola vez y de inmediato (QAS-07), el cierre de partida debe ser atómico (QAS-06) y el equipo necesita probar las reglas sin infraestructura (QAS-10). Están en juego ASR-01, ASR-05 y ASR-06. |
| **Alternativas** | A. Cliente-servidor en capas con servidor autoritativo: el cliente envía intenciones; el servidor valida con un dominio sin dependencias y guarda a través de repositorios. A favor: un solo lugar decide; el dominio se prueba solo y una IA futura podría reutilizarlo (DES-09); solo el servidor toca la base. En contra: cada jugada cuesta un viaje de ida y vuelta; el servidor es punto único de falla (R-04).<br>B. Igual a igual, con el cliente de un jugador como anfitrión de la partida. A favor: sin servidor para jugar; menor latencia para el anfitrión. En contra: el anfitrión puede hacer trampa, lo que invalida el ranking; el elo, el ranking, los amigos y los reportes igual necesitan un servidor central.<br>C. Reglas en el cliente y servidor como simple relé y almacén. A favor: respuesta inmediata; servidor más sencillo. En contra: un cliente modificado decide el resultado y puede saltarse el filtro del chat; contradice CU-13 RN-01 y CU-21 RN-01. |
| **Criterios** | Integridad frente a clientes modificados (ASR-01), atomicidad de lo que se guarda (ASR-05), capacidad de prueba y reutilización de reglas (ASR-06), esfuerzo para dos personas. |
| **Decisión** | A. Patrón cliente-servidor con arquitectura en capas: cliente (presentación) → contratos; host → servicios → dominio; servicios → acceso a datos → base de datos. Patrón repositorio (`IXRepository`) entre servicios y datos. El cliente recibe el estado por callbacks y lo calcula solo para dar respuesta inmediata, nunca para decidir. Lo que describe a jugadores conectados y no se guarda (la cola de emparejamiento, las plazas y los expulsados de cada sala, los emotes) lo lleva el servidor en memoria (D-24, D-27, D-10). |
| **Consecuencias** | Se gana un único punto de verdad, pruebas del dominio y de los servicios con dobles, y reglas de dependencia comprobables. Se paga la latencia de ida y vuelta en cada jugada (QAS-03), la dependencia de un único servidor y que un reinicio vacíe la cola y las salas. |
| **Riesgos** | R-04, R-05 |
| **Elementos afectados** | C-01, C-02, C-03, C-04, C-05, C-06 |

| **ADR-002 · Comunicar cliente y servidor con CoreWCF dúplex sobre net.tcp y llamadas asíncronas** | |
|---|---|
| **Estado** | Aceptado |
| **Contexto** | El curso pide comunicación dúplex sobre net.tcp con llamadas asíncronas y prohíbe stack web, WebSockets, SignalR, gRPC y REST. Los cambios de partida, sala, chat y emparejamiento, y las invitaciones de los amigos, deben llegar a los clientes en menos de 500 ms (QAS-03) y todo debe correr en macOS y en Windows (QAS-08). Están en juego ASR-02 y ASR-03. |
| **Alternativas** | A. CoreWCF 1.9 sobre .NET 10, `NetTcpBinding` dúplex, contratos en una biblioteca netstandard2.0 y cliente con `System.ServiceModel`. A favor: corre en ambos sistemas; callbacks en el mismo canal; contratos compartidos. En contra: CoreWCF no implementa todo WCF; usa reflexión, por lo que no se puede recortar ni compilar AOT.<br>B. WCF clásico sobre .NET Framework 4.8. A favor: es lo que se ve en clase; implementación completa. En contra: solo Windows, de modo que el servidor viviría en una sola máquina del equipo (punto único de falla para el desarrollo) y el integrante de macOS no podría ejecutarlo.<br>C. CoreWCF de petición-respuesta con sondeo periódico del cliente. A favor: contratos más simples, sin callbacks. En contra: la latencia depende del intervalo de sondeo (incumple QAS-03 si es largo, sobrecarga el servidor si es corto); cada cliente que espera en una sala, en la cola o una invitación tendría que sondear sin parar. |
| **Criterios** | Tecnologías permitidas, entrega de cambios a los clientes (ASR-02), multiplataforma (ASR-03), latencia y carga. |
| **Decisión** | A. Contratos de servicio y de callback en `Bastion.Contracts` (netstandard2.0), operaciones asíncronas en el cliente y difusión por callbacks en el servidor. `PublishTrimmed` y `PublishAot` quedan desactivados en `Directory.Build.props`. |
| **Consecuencias** | Se gana notificación inmediata y un solo canal por cliente. Se paga un canal con estado que hay que vigilar (fallas del canal = desconexión, CU-20). |
| **Riesgos** | R-05, R-08 |
| **Elementos afectados** | C-01, C-02, C-03, C-04 |

| **ADR-003 · Construir el cliente con MonoGame DesktopGL** | |
|---|---|
| **Estado** | Aceptado |
| **Contexto** | El cliente es un juego de escritorio que debe correr en macOS arm64 y en Windows (ASR-03, QAS-08), dibujar el tablero y las pantallas, y ser código C# que el equipo escribe y que se evalúa. |
| **Alternativas** | A. MonoGame DesktopGL 3.8.5. A favor: biblioteca de código, corre en macOS y Windows con OpenGL; todo lo hace el código C# del equipo. En contra: no trae controles de interfaz, por lo que el equipo construye los suyos.<br>B. Godot con C#. A favor: editor de escenas y controles de interfaz listos. En contra: buena parte del trabajo se hace en su editor y no en código, que es lo evaluado.<br>C. Unity. A favor: motor completo y documentación amplia. En contra: sobredimensionado para un tablero 2D; proyecto pesado de versionar entre dos personas.<br>D. Stride. A favor: motor C# de código abierto. En contra: su editor solo funciona en Windows.<br>E. Avalonia. A favor: interfaz de escritorio multiplataforma en C#. En contra: no es un motor de juego: sin bucle de juego ni animación del tablero.<br>F. MonoGame WindowsDX. A favor: DirectX nativo. En contra: solo Windows. |
| **Criterios** | Multiplataforma (ASR-03), que el trabajo esté en código evaluable, peso del proyecto y curva de aprendizaje. |
| **Decisión** | A. El cliente es `Bastion.Client` con MonoGame DesktopGL, un conjunto propio de controles y pantallas, y un navegador de pantallas. |
| **Consecuencias** | Se gana control total y un cliente idéntico en ambos sistemas. Se paga construir controles (campos de texto, listas, ventanas emergentes) y decidir cómo se presenta el tablero (R-02). |
| **Riesgos** | R-02 |
| **Elementos afectados** | C-01 |

| **ADR-004 · Usar SecurityMode.None y concentrar la seguridad en el servidor** | |
|---|---|
| **Estado** | Sustituido por ADR-007. Se conserva como registro: las medidas del servidor que describe siguen vigentes; lo que cambia es que el canal ya no viaja en claro. |
| **Contexto** | La seguridad por omisión de net.tcp usa autenticación de Windows, que no funciona desde macOS (ASR-03). Aun así hay que proteger credenciales y partidas (QAS-01, QAS-02, ASR-04). |
| **Alternativas** | A. `SecurityMode.None` compensado en el servidor: validación de todo, contraseñas PBKDF2-SHA512 con 210 000 iteraciones y sal por cuenta, comparación en tiempo constante, mismo resultado ante cualquier fallo, bloqueo escalonado, token de sesión del que solo se guarda su SHA-256, bitácora de accesos y base accesible solo con un usuario sin `DELETE`. A favor: funciona entre Mac y Windows; protege lo que se guarda. En contra: lo que viaja por la red va en claro.<br>B. Seguridad de transporte con autenticación de Windows (por omisión). A favor: cifrado sin configuración en Windows. En contra: no funciona desde macOS.<br>C. Seguridad de transporte con certificado TLS. A favor: cifra el canal. En contra: hay que emitir y distribuir certificados a cada cliente y no se ha comprobado que el cliente de .NET en macOS lo soporte con CoreWCF. |
| **Criterios** | Funcionar entre ambos sistemas (ASR-03), proteger credenciales y estado (ASR-04), esfuerzo de despliegue. |
| **Decisión** | A, con el supuesto de una red de confianza (sección 2.4). La alternativa C queda como mejora si el supuesto resulta falso. |
| **Consecuencias** | Se gana un sistema que funciona en ambos sistemas y que no guarda secretos en claro. Se paga que un atacante en la misma red puede leer contraseñas y tokens en tránsito. |
| **Riesgos** | R-01 |
| **Elementos afectados** | C-02, C-03, C-04, C-06, C-07 |

| **ADR-005 · Persistir con EF Core y SQL Server 2022, con el esquema en scripts SQL** | |
|---|---|
| **Estado** | Aceptado |
| **Contexto** | Las operaciones de partida, sala y moderación deben ser atómicas (ASR-05), la base debe correr en las dos máquinas (ASR-03) y los datos de las cuentas deben quedar protegidos (ASR-04). Los scripts SQL del modelo de datos ya existen en los requisitos. |
| **Alternativas** | A. EF Core 10 con el proveedor de SQL Server, sin migraciones: los scripts SQL mandan y el código se ajusta a ellos; SQL Server 2022 en Docker. A favor: transacciones y consultas tipadas; un solo origen del esquema. En contra: hay que mantener a mano la correspondencia entre entidades y tablas.<br>B. EF Core con migraciones. A favor: el esquema se genera desde el código. En contra: dos fuentes de verdad, los scripts y las migraciones, que tarde o temprano divergen.<br>C. ADO.NET con SQL escrito a mano. A favor: control total y lo más cercano a lo visto en clase. En contra: mucho más código repetitivo y propenso a errores de mapeo.<br>D. Azure SQL Edge en Docker para tener imagen ARM en Mac. A favor: nativo en Apple silicon. En contra: Microsoft lo retiró. |
| **Criterios** | Atomicidad (ASR-05), multiplataforma (ASR-03), un solo origen del esquema, esfuerzo. |
| **Decisión** | A. `BastionDbContext` con configuraciones por entidad y repositorios en `Bastion.Server.Data`; el inicio de sesión `BastionServerConnection` no tiene permiso `DELETE`; la cadena de conexión vive en user-secrets o en variables de entorno, nunca en el repositorio. En Mac la imagen x64 corre con Rosetta. |
| **Consecuencias** | Se gana transacciones con poco código y un esquema único. Se paga la dependencia de Docker y Rosetta en Mac. |
| **Riesgos** | R-03 |
| **Elementos afectados** | C-06, C-07 |

| **ADR-006 · Traducir en el cliente los resultados codificados del servidor** | |
|---|---|
| **Estado** | Aceptado |
| **Contexto** | El juego se ofrece en en-US y es-MX (QAS-11); todo texto visible, incluidos los errores, debe mostrarse en el idioma elegido y agregar un idioma no debe requerir cambiar código (ASR-07, CU-01 RN-12, D-21). |
| **Alternativas** | A. El servidor devuelve enumeraciones de resultado; el cliente las convierte en una clave `Pantalla.Elemento` y busca el texto en `Strings.resx` (en-US, neutral y por omisión) o `Strings.es-MX.resx`. A favor: el contrato no cambia con el idioma; agregar un idioma solo agrega un archivo. En contra: hay que mantener la correspondencia código–clave.<br>B. El servidor genera el mensaje ya traducido según el idioma de la sesión. A favor: el cliente solo lo muestra. En contra: el texto viaja en el contrato; mezclar presentación en el servidor; el cliente no puede adaptar el mensaje a su pantalla.<br>C. Traducciones guardadas en la base de datos. A favor: se cambian sin recompilar. En contra: D-21 lo descarta; más consultas y más tablas sin necesidad. |
| **Criterios** | Idioma de todos los textos (ASR-07), contratos estables, esfuerzo para agregar idiomas. |
| **Decisión** | A. El idioma por omisión es en-US, lo que sustituye el es-MX por omisión previsto en D-21. Los correos, que salen del servidor, usan los recursos del servidor con el idioma preferido de la cuenta (D-21). |
| **Consecuencias** | Se gana un contrato independiente del idioma y una prueba simple de claves faltantes. Se paga mantener dos archivos de recursos sincronizados. |
| **Riesgos** | R-07 |
| **Elementos afectados** | C-01, C-02, C-04 |

| **ADR-007 · Cifrar el canal net.tcp con TLS y reconocer al servidor por la huella de su certificado** | |
|---|---|
| **Estado** | Aceptado. Sustituye a ADR-004. |
| **Contexto** | Con ADR-004 las contraseñas, los códigos del segundo factor y los tokens de sesión viajaban en claro (R-01 en la v0.1), lo que contradice CU-01 RN-02 y la preocupación CRN-22. La solución debe seguir funcionando entre macOS y Windows, sin autenticación de Windows (ASR-03). Están en juego ASR-04 y ASR-03. |
| **Alternativas** | A. `SecurityMode.Transport` con `ClientCredentialType = None`: TLS con un certificado del servidor; el cliente no presenta credenciales de transporte, porque se identifica con su cuenta, y acepta el certificado solo si su huella SHA-256 coincide con la configurada. A favor: cifra todo el canal; funciona desde macOS porque no interviene Windows; basta un certificado autofirmado, sin autoridad certificadora. En contra: hay que generar el certificado, guardarlo fuera del repositorio y llevar su huella a cada cliente; cambiarlo obliga a actualizar la huella.<br>B. `SecurityMode.Transport` validando el certificado contra los almacenes de confianza del sistema. A favor: el certificado se renueva sin tocar los clientes. En contra: exige un certificado emitido por una autoridad o instalar una raíz propia en cada máquina, con mecanismos distintos en macOS y en Windows; impráctico para un servidor de aula sin nombre de dominio.<br>C. TLS mutuo (`ClientCredentialType = Certificate`). A favor: el servidor también autentica al equipo cliente. En contra: un certificado por cliente que emitir y distribuir, cuando la identidad ya la da la cuenta del jugador.<br>D. Mantener `SecurityMode.None` y cifrar en la aplicación los campos sensibles. A favor: sin certificados. En contra: criptografía propia, propensa a errores; el token de sesión y el chat seguirían en claro. |
| **Criterios** | Que nada sensible viaje en claro (ASR-04, CU-01 RN-02), funcionar entre ambos sistemas (ASR-03), esfuerzo de despliegue para dos personas en un aula. |
| **Decisión** | A. Cliente y servidor usan `NetTcpBinding(SecurityMode.Transport)` con `TcpClientCredentialType.None`. El servidor carga su certificado PKCS#12 desde `Server:CertificatePath`, con la contraseña en `Server:CertificatePassword` (user-secrets), y no arranca sin él. El cliente solo confía en el servidor si la huella SHA-256 del certificado coincide con la variable de entorno `BASTION_SERVER_THUMBPRINT`. El certificado y su contraseña nunca entran al repositorio. Se conservan las medidas del servidor descritas en ADR-004: PBKDF2-SHA512, comparación en tiempo constante, bloqueo escalonado, token del que solo se guarda su resumen, bitácora de accesos y usuario de base de datos sin `DELETE`. |
| **Consecuencias** | Se gana un canal cifrado de punta a punta que funciona igual en los dos sistemas. Se paga administrar un certificado: generarlo, guardar su contraseña fuera del repositorio y avisar a cada cliente de la nueva huella cada vez que cambie; un cliente sin la huella configurada no se conecta. |
| **Riesgos** | R-01 |
| **Elementos afectados** | C-01, C-03 |

## 4. Estructura

Primera entrega inicia · Entrega final completa

*Describan los elementos y sus reglas con texto y tablas. Los diagramas van en la sección 6 y usan estos mismos identificadores.*

## 4.1 Módulos y componentes

| ID | Nombre y tipo | Responsabilidad | Provee / requiere |
|---|---|---|---|
| C-01 | Bastion.Client · módulo (ejecutable de escritorio) | Dibuja las pantallas y el tablero, captura la entrada del jugador, muestra los textos en en-US o es-MX y llama a los servicios del servidor tras validar la huella de su certificado; no decide ninguna regla. | Requiere los contratos de servicio de C-02; provee los contratos de callback de C-02. |
| C-02 | Bastion.Contracts · módulo (biblioteca netstandard2.0) | Define los contratos de servicio y de callback, los datos que viajan y los resultados codificados. | Provee `IAccountService` e `ILeaderboardService`, ya construidos, y los contratos por construir de partida, sala, chat, amigos y reportes; no requiere nada. |
| C-03 | Bastion.Server.Host · componente (proceso del servidor) | Raíz de composición: levanta CoreWCF sobre net.tcp con TLS (ADR-007), registra servicios y repositorios en el contenedor de dependencias, lee la configuración y los secretos, entre ellos el certificado, y configura log4net. | Provee los extremos net.tcp de C-02; requiere C-04 y C-06. |
| C-04 | Bastion.Server.Services · módulo | Implementa los contratos: valida cada solicitud, aplica la seguridad (resúmenes de contraseña, tokens, bloqueo, segundo factor, auditoría), coordina partidas y transacciones, aplica los strikes, lleva en memoria la cola de emparejamiento y las salas (D-24, D-27) y difunde por callbacks. | Provee las implementaciones de C-02; requiere C-05, las interfaces `IXRepository` de C-06 y la interfaz de C-08. |
| C-05 | Bastion.Domain · módulo | Reglas del juego: tablero, peones, muros, saltos, turnos y validación de caminos. | Provee la API de reglas; no requiere nada. |
| C-06 | Bastion.Server.Data · módulo | Acceso a datos con EF Core: contexto, entidades, configuraciones y repositorios. | Provee `IAccountRepository`, `ISessionRepository`, `IAccessLogRepository`, `ILeaderboardRepository` y `IGameModeRepository`; requiere C-07. |
| C-07 | Base de datos Bastion · componente (SQL Server 2022 en Docker) | Guarda cuentas, sesiones, códigos de verificación, partidas con sus jugadas, salas, mensajes, amistades, reportes, sanciones y la bitácora de accesos; garantiza unicidad y reglas con índices y restricciones. | Provee el esquema de los scripts de `database/`; se accede solo con `BastionServerConnection`. |
| C-08 | Pasarela de correo y segundo factor · componente (planeado) | Envía el correo de verificación de la cuenta y los avisos de seguridad (cambio de contraseña, activación y desactivación del segundo factor), y entrega el segundo factor, detrás de una interfaz. | Provee una interfaz de envío a C-04; requiere el servicio de correo externo. |

*Lo repetitivo se resuelve por convención: escriban la convención una vez, debajo de la tabla.*

**Convención.** Dentro de cada módulo el código se agrupa por funcionalidad (`Accounts`, `Leaderboard`, `Sessions`...) y el espacio de nombres sigue la carpeta; cada tipo va en su propio archivo con su nombre. Cada funcionalidad repite el mismo trío: un contrato `IXService` en C-02, su implementación `XService` en C-04 y uno o más `IXRepository` con su implementación en C-06. Las entidades de EF Core nunca salen de C-04: se convierten en datos del contrato. Las pantallas del cliente siguen el par `XScreen` y `XController`, y sus textos usan claves `Pantalla.Elemento`.

### 4.2 Interfaces críticas

*Sólo las interfaces cuyo incumplimiento produce un riesgo real: dinero, estado compartido, datos personales...*

| Interfaz | Provista por / usada por | Operaciones | Contrato: condiciones de entrada, resultado garantizado y errores relevantes |
|---|---|---|---|
| *[Nombre]* | *[C-__ / C-__]* | *[...]* | *[...]* |
| | | | |

### 4.3 Reglas de dependencia

| Regla | Permitida o prohibida | Por qué (ADR) |
|---|---|---|
| *[...]* | *[...]* | *[ADR-__]* |
| | | |

## 5. Arquitectura en ejecución Entrega final

*Cada preocupación tiene una decisión, o un «no aplica» con su razón. No es una lista de tecnologías: toda tecnología mencionada se conecta con un ADR o un ASR.*

| Preocupación | Decisión, o «no aplica porque...» | ADR / ASR | Vista o evidencia |
|---|---|---|---|
| **Distribución y comunicación** | *[...]* | *[...]* | *[V-__ / EV-__]* |
| **Datos, estado y propiedad** | *[...]* | *[...]* | *[V-__ / EV-__]* |
| **Concurrencia y coordinación** | *[...]* | *[...]* | *[V-__ / EV-__]* |
| **Resiliencia y disponibilidad** | *[...]* | *[...]* | *[V-__ / EV-__]* |
| **Seguridad y límites de confianza** | *[...]* | *[...]* | *[V-__ / EV-__]* |
| **Despliegue y entornos** | *[...]* | *[...]* | *[V-__ / EV-__]* |
| **Observabilidad y operación** | *[...]* | *[...]* | *[V-__ / EV-__]* |
| **Costo operacional** | *[...]* | *[...]* | *[V-__ / EV-__]* |

# 6. Vistas

Primera entrega: V-01 y V-02 | Entrega final completa

*Una vista existe para responder a una audiencia concreta. Usen la notación que mejor sirva a ese propósito; en el curso se practican UML y C4.*

## 6.1 Tipos de vista que usaremos

*En ISO 42010 estos tipos se llaman viewpoints. No tienen que inventarlos: elijan los que necesiten. Agreguen otro sólo si una preocupación importante queda sin atender.*

| Tipo de vista | Qué preocupaciones atiende | Representaciones útiles |
|---|---|---|
| **Contexto** | Frontera del sistema, usuarios y sistemas externos | Diagrama de contexto; C4 nivel 1 |
| **Estructura** | Módulos, componentes, interfaces y dependencias | UML de componentes o de paquetes; C4 niveles 2 y 3 |
| **Ejecución** | Interacciones, estado compartido, concurrencia y fallas | UML de secuencia, de actividad o máquina de estados |
| **Despliegue** | Nodos, comunicación, límites de confianza y redundancia | UML de despliegue |

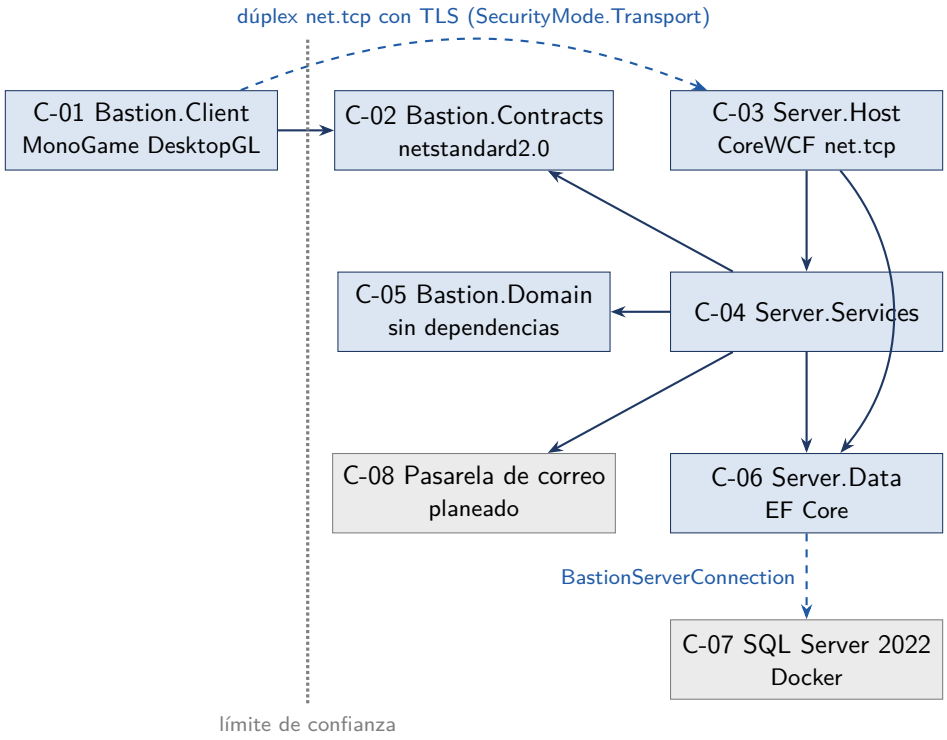
## 6.2 Vistas

*Copien la ficha para cada vista y coloquen el diagrama justo debajo. Declaren al menos una correspondencia con otra vista: qué debe coincidir entre ambas.*

| **V-01 · Contexto de Bastión** | |
|---|---|
| **Tipo de vista** | Contexto |
| **Para quién y para qué** | Para el profesor y el equipo: ver de un vistazo qué está dentro del sistema, quién lo usa y de qué sistemas externos depende. |
| **Preocupaciones que atiende** | Frontera del sistema, actores humanos y dependencias externas (sección 2.1). |
| **Notación y leyenda** | C4 nivel 1. Figuras: personas; recuadro azul: el sistema; recuadros grises: sistemas externos; flechas: quién inicia la comunicación. |
| **Se corresponde con** | V-02: el recuadro de Bastión se descompone en C-01 a C-08, y las flechas hacia el correo y el segundo factor salen de C-08. |



| V-02 · Estructura en capas | |
|---|---|
| **Tipo de vista** | Estructura |
| **Para quién y para qué** | Para el equipo: saber en qué módulo va cada clase y qué referencias están permitidas; para el profesor: comprobar que la estructura sigue ADR-001. |
| **Preocupaciones que atiende** | Módulos, dependencias permitidas, frontera de red y quién accede a la base de datos. |
| **Notación y leyenda** | UML de componentes simplificado. Recuadros: módulos o componentes con su C-xx; flecha continua: dependencia de compilación; flecha discontinua: comunicación por red o por conexión a la base; línea punteada: límite de confianza entre cliente y servidor. |
| **Se corresponde con** | V-01: C-01 a C-08 forman el recuadro de Bastión y C-08 es la única salida hacia el correo y el segundo factor; la tabla 4.1: cada recuadro es una fila con el mismo identificador. |




# 7. Evaluación, trazabilidad y transición

Primera entrega: sólo 7.3 Entrega final

*Muestren con evidencia qué tan bien responde la arquitectura a sus ASR, qué queda pendiente y qué necesita saber quien la construya.*

## 7.1 Evaluación y trazabilidad de los ASR

Entrega final

*Una fila por ASR. Esta tabla es la matriz de trazabilidad: no la repitan en otro lugar. En «Resultado» no escriban sólo «cumple»: digan qué mostró la evidencia.*

| ASR | ADR | Elementos | Vista | Evidencia | Resultado | Riesgo residual |
|---|---|---|---|---|---|---|
| ASR-01 | *[ADR-___]* | *[C-___]* | *[V-___]* | *[EV-___]* | *[Qué mostró la evidencia]* | *[R-___]* |
| | | | | | | |

## 7.2 Evidencias Entrega final

*Registren aquí cada EV que citen. Una evidencia puede ser una revisión por escenario, una medición, una prueba o un análisis. Es obligatorio incluir al menos una verificación empírica ejecutada y una comprobación de conformidad.*

| EV | Tipo | Qué se hizo y qué resultó | Archivo o referencia |
|---|---|---|---|
| EV-01 | *[Revisión / Medición / Prueba / Análisis]* | *[. . . ]* | *[evidencia/. . . ]* |
| | | | |

| **Verificación empírica · EV-___** | |
|---|---|
| **Afirmación que se pone a prueba** | *[Qué esperan de la arquitectura]* |
| **Cómo se probó** | *[Prototipo pequeño (spike), medición o prueba; condiciones]* |
| **Resultado** | *[Qué se observó, con números]* |
| **Qué cambia en la arquitectura** | *[Se confirma, se matiza o se refuta; ajuste realizado]* |

> **No necesitan construir el sistema completo:** un prototipo pequeño basta si pone a prueba la afirmación.

| **Comprobación de conformidad · EV-___** | |
|---|---|
| **Decisión que protege** | *[ADR-___ o regla de dependencia]* |
| **Procedimiento** | *[Pasos que otra persona puede repetir. Automatizarlo es deseable, no obligatorio]* |
| **Resultado esperado** | *[Cuándo se considera que la regla se violó]* |

## 7.3 Riesgos y pendientes Primera entrega inicia

*Se inicia en la primera entrega y se mantiene hasta la entrega final.*

| R | Tipo | Descripción | Qué se hará |
|---|---|---|---|
| R-01 | Riesgo | El canal va cifrado con TLS (ADR-007), pero si el certificado del servidor y su contraseña se filtran, alguien podría hacerse pasar por el servidor; además, cada cambio de certificado obliga a actualizar la huella en todos los clientes. En la v0.1 este riesgo era el tráfico en claro de ADR-004, ya resuelto. | Guardar el certificado y su contraseña solo en las máquinas del equipo (user-secrets), nunca en el repositorio; documentar cómo generarlo y obtener su huella; cambiarlo si se sospecha una filtración. |

| R | Tipo | Descripción | Qué se hará |
|---|---|---|---|
| R-02 | Decisión abierta | Presentación del tablero: plano, isométrico 2D o 3D (ADR-003). | Decidir con un prototipo de cada opción en MonoGame antes de construir la pantalla de partida; se registrará como ADR. |
| R-03 | Riesgo | En macOS arm64 no hay imagen ARM de SQL Server 2022; la imagen x64 corre con Rosetta, con posible pérdida de rendimiento o fallas (ADR-005). | Medir el tiempo de las consultas de QAS-04 en Mac; si falla, usar el servidor de base de la máquina Windows durante el desarrollo. |
| R-04 | Riesgo | Un único servidor guarda en memoria la cola de emparejamiento, las salas con sus expulsados y las partidas en curso: si se cae, ese estado se pierde y el servicio se interrumpe (ADR-001, QAS-05, D-24, D-27). | Escribir cada jugada antes de notificarla (ASR-05) para reconstruir partidas desde la base; aceptar el punto único de falla dado el supuesto de uso en aula. |
| R-05 | Riesgo | No se ha medido la latencia de los callbacks con 20 partidas simultáneas, ahora con el costo añadido del cifrado TLS (QAS-03, ADR-007). | Construir un prototipo de carga con clientes simulados para la entrega final (será una EV). |
| R-06 | Decisión abierta | El repositorio remoto será GitLab o Bitbucket. | Elegir según lo que indique el curso; no afecta la arquitectura. |
| R-07 | Pendiente | No se ha elegido el servicio de correo ni el proveedor del segundo factor, y los correos aún no tienen recursos por idioma (C-08, ADR-006). | Definir la interfaz en C-04 y una implementación de prueba que escriba en la bitácora; elegir el proveedor antes de construir CU-02 y CU-04 a CU-06. |
| R-08 | Deuda | CoreWCF depende de reflexión y arrastra dependencias con vulnerabilidades conocidas: una se fijó a mano (`System.Security.Cryptography.Xml`) y nunca debe activarse `PublishTrimmed` ni `PublishAot` (ADR-002). | Revisar los avisos de paquetes en cada entrega; mantener la nota en `Directory.Build.props`. |
| R-09 | Pendiente | El esquema de la base de datos y el código aún reflejan el alcance anterior al recorte (D-25): los scripts conservan tablas que solo usan requisitos deseables (silencios, bloqueos, apelaciones, bitácora de moderación) y les faltan las de amistades, solicitudes e invitaciones; el cliente conserva pantallas de deseables (tienda, espectador, panel de moderación). | Ajustar los scripts de los requisitos y de `database/` al alcance de 28 casos antes de construir CU-21 a CU-28, y retirar del cliente las pantallas que ningún caso del alcance usa. |

## 7.4 Transición a Construcción Entrega final

*Instrucciones accionables para quien implemente: decisiones que no deben romperse, interfaces críticas, comprobaciones que deben ejecutarse y orden sugerido de trabajo. Si ya hay código, registren las diferencias con el diseño; si no, cómo se comprobarán.*

*[Guía breve.]*

| Elemento o ADR | Qué dice el diseño | Qué hay en el código, o cómo se comprobará | Qué se hará |
|---|---|---|---|
| *[ADR-___ / C-___]* | *[. . . ]* | *[. . . ]* | *[. . . ]* |
| | | | |

## Anexos

*Opcionales y sólo para material extenso. Una decisión nueva no puede aparecer únicamente en un anexo.*

## Lista de comprobación · Primera entrega

- ☐ Contexto, interesados con sus preocupaciones y funciones clave identificados.
- ☐ Restricciones, supuestos, objetivos y condiciones separados por tipo.
- ☐ Entre 8 y 12 escenarios medibles; al menos uno de seguridad con activo, actor de amenaza y límite de confianza.
- ☐ Árbol de utilidad priorizado por importancia y dificultad.
- ☐ Entre 5 y 7 ASR, cada uno con su procedencia.
- ☐ Entre 4 y 6 ADR con estado; cada uno compara al menos dos alternativas reales.
- ☐ Elementos principales en 4.1 y vistas V-01 (contexto) y V-02 (estructura).
- ☐ Riesgos y pendientes iniciados en 7.3; declaración de IA actualizada.

## Lista de comprobación · Entrega final

- ☐ Las siete secciones están presentes y no hay relleno sin propósito.
- ☐ Interfaces críticas con contrato y reglas de dependencia declaradas.
- ☐ Cada preocupación de ejecución tiene una decisión o un «no aplica» justificado.
- ☐ Cada vista declara audiencia, propósito y al menos una correspondencia.
- ☐ La tabla 7.1 permite seguir cada ASR hasta su evidencia y su riesgo residual.
- ☐ Hay al menos una verificación empírica ejecutada y una comprobación de conformidad repetible.
- ☐ Hay diferencias registradas con el código, o un plan para comprobarlas, y una guía de transición.
- ☐ Ambos integrantes pueden explicar cualquier decisión del documento.

> **Entrega.** El paquete se arma conforme al Manual vigente. El PDF de este documento es la versión que se califica.